Bag-of-Words

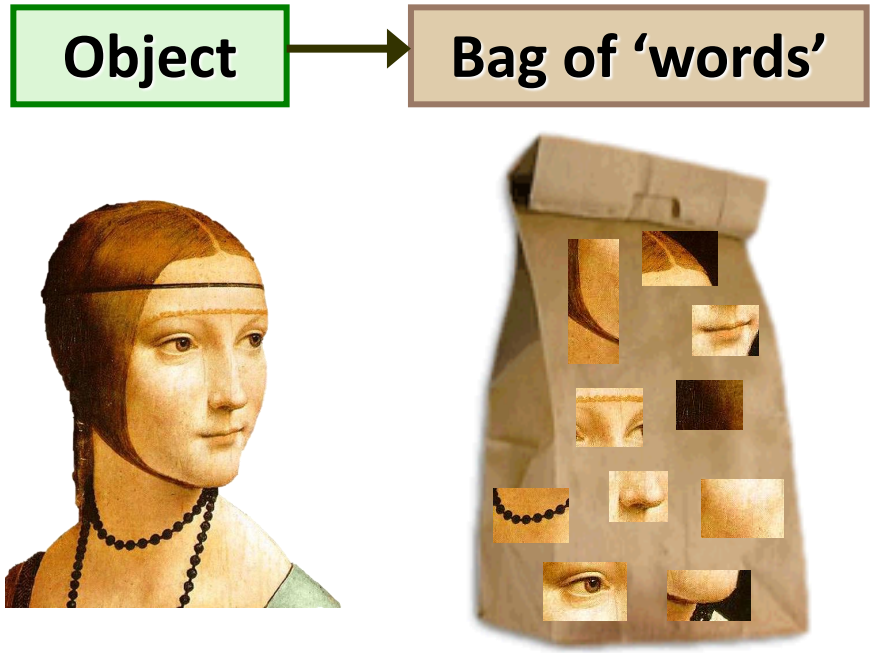


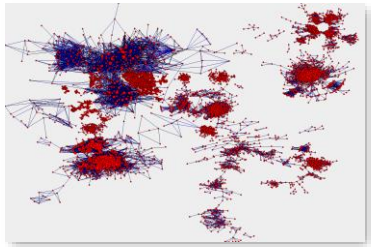
Image matching

- Brute force approach:

- 250,000 images → ~ 31 billion image pairs
  - 2 pairs per second → 1 year on 500 machines

- 1,000,000 images → 500 billion pairs
  - 15 years on 500 machines

Image matching

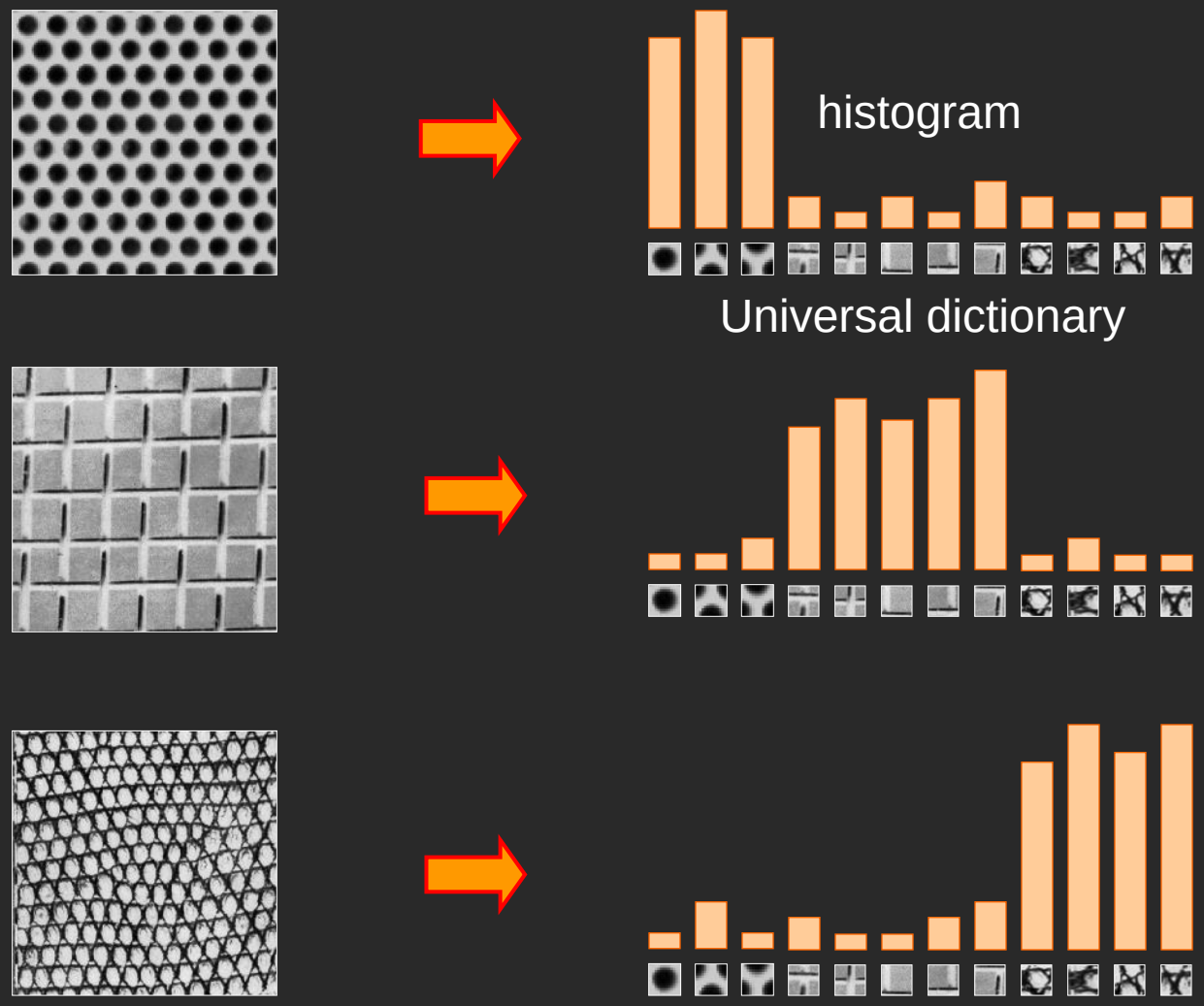


- For city-sized datasets, fewer than 0.1% of image pairs actually match

- Key idea: only consider *likely* matches
- How do we know if a match is likely?
- Solution: use fast global similarity measures
  - For example, a *bag-of-words* representation

Object → **Bag of 'words'**

Origin 1: Texture recognition



Julesz, 1981; Cula & Dana, 2001; Leung & Malik 2001; Mori, Belongie & Malik, 2001; Schmid 2001; Varma & Zisserman, 2002, 2003; Lazebnik, Schmid & Ponce, 2003

Origin 2: Bag-of-words models

- Orderless document representation: frequencies of words from a dictionary Salton & McGill (1983)

Origin 2: Bag-of-words models

- Orderless document representation: frequencies of words from a dictionary Salton & McGill (1983)

US Presidential Speeches Tag Cloud
http://chir.ag/phernalia/preztags/

Origin 2: Bag-of-words models

- Orderless document representation: frequencies of words from a dictionary Salton & McGill (1983)

US Presidential Speeches Tag Cloud
http://chir.ag/phernalia/preztags/

Origin 2: Bag-of-words models

- Orderless document representation: frequencies of words from a dictionary Salton & McGill (1983)

US Presidential Speeches Tag Cloud
http://chir.ag/phernalia/preztags/

Origin 2: Bag-of-words models

John likes to watch movies. Mary likes too.

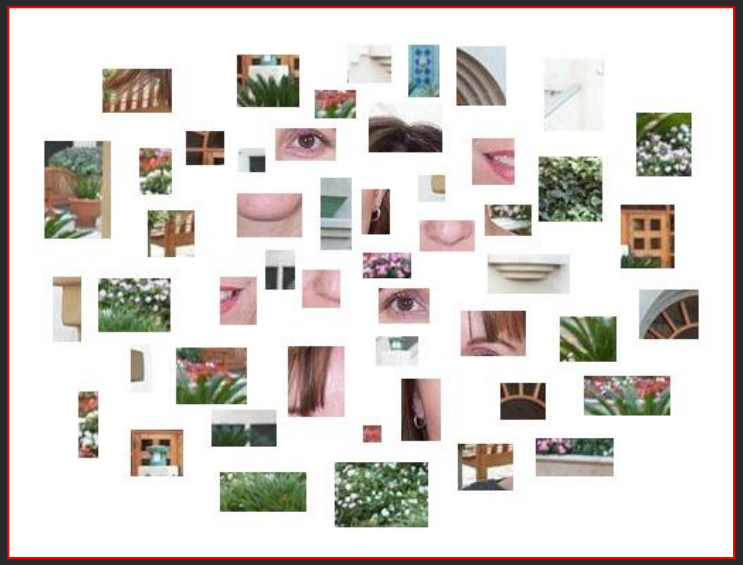
John also likes to watch football games

{"John": 1, "likes": 2, "to": 3, "watch": 4, "movies": 5,
"also": 6, "football": 7, "games": 8, "Mary": 9, "too": 10}

[1, 2, 1, 1, 1, 0, 0, 0, 1, 1]

[1, 1, 1, 1, 0, 1, 1, 1, 0, 0]

Bag of words

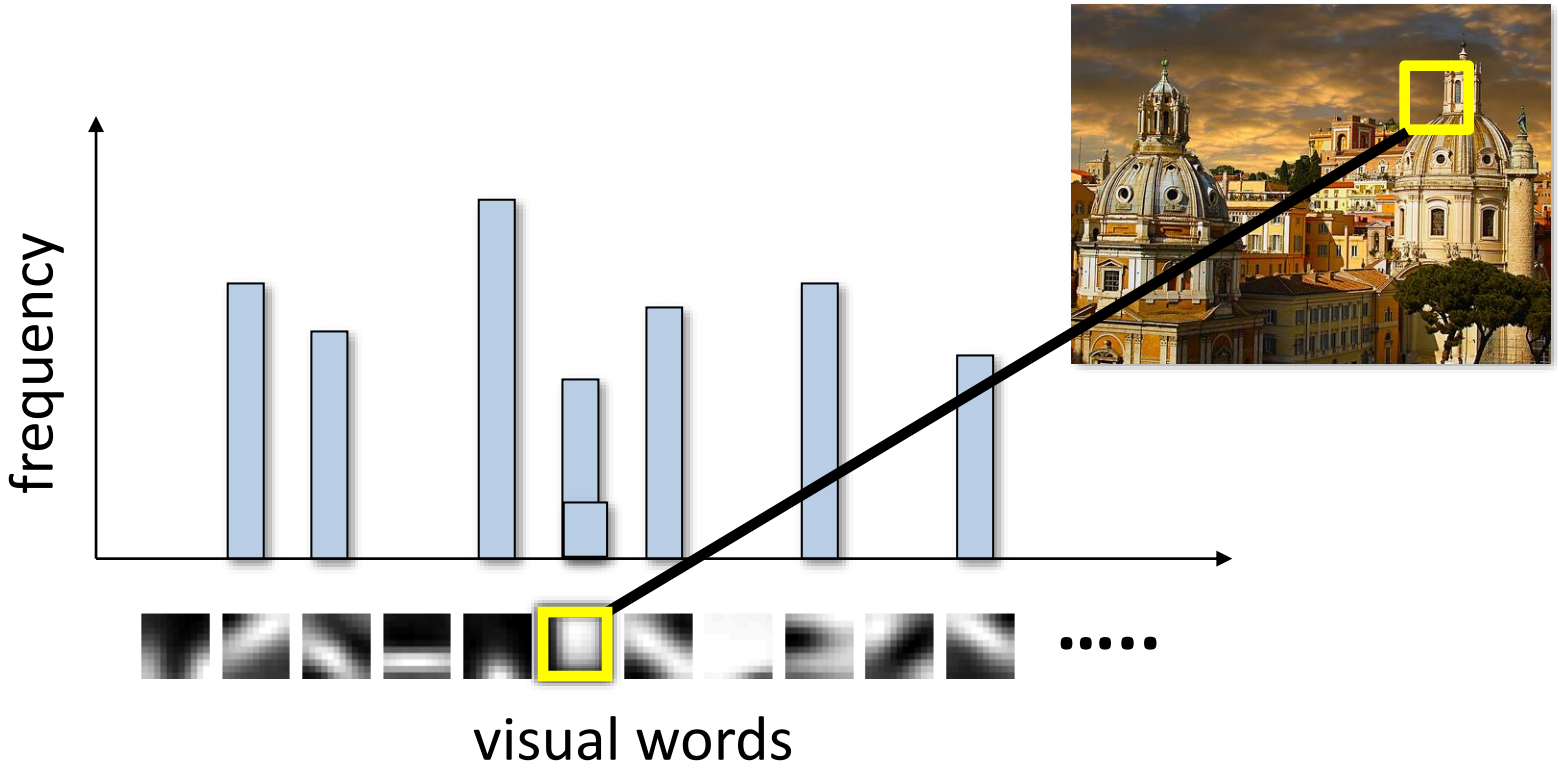


face, flowers, building

Works pretty well image retrieval, recognition and matching

Images as histograms of visual words

- Inspired by ideas from text retrieval
  - [Sivic and Zisserman, ICCV 2003]

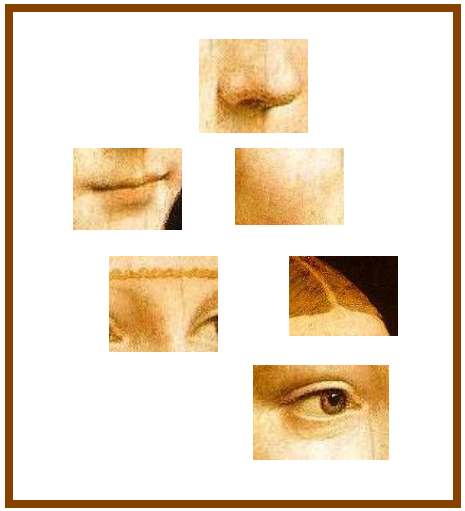


Quiz: What is BoW for one image?

- A histogram of local feature vectors in an image
- A visual dictionary
- The feature vector of a local image patch
- A histogram of local features in the collection of images

Bag of features: outline

1. Extract features



Bag of features: outline

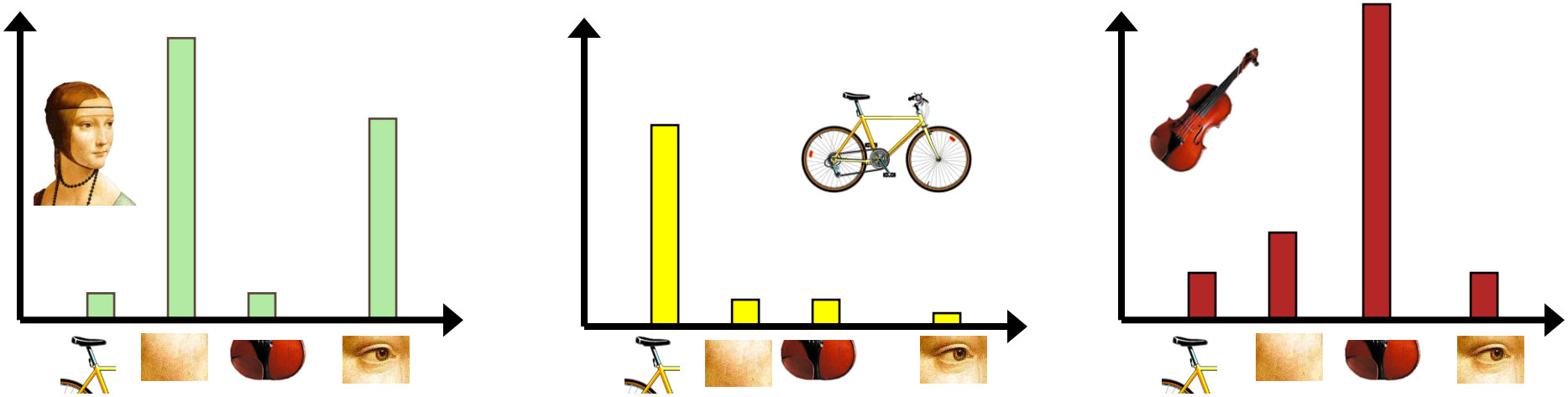
1. Extract features
2. Learn “visual vocabulary”

Bag of features: outline

1. Extract features
2. Learn “visual vocabulary”
3. Quantize features using visual vocabulary

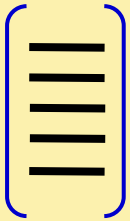
Bag of features: outline

1. Extract features
2. Learn “visual vocabulary”
3. Quantize features using visual vocabulary
4. Represent images by frequencies of “visual words”



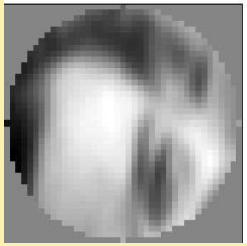
Quantize: approximate by one whose amplitude is restricted to a prescribed set of values.

1. Feature extraction



Compute SIFT descriptor

[Lowe'99]



Normalize patch

Detect patches

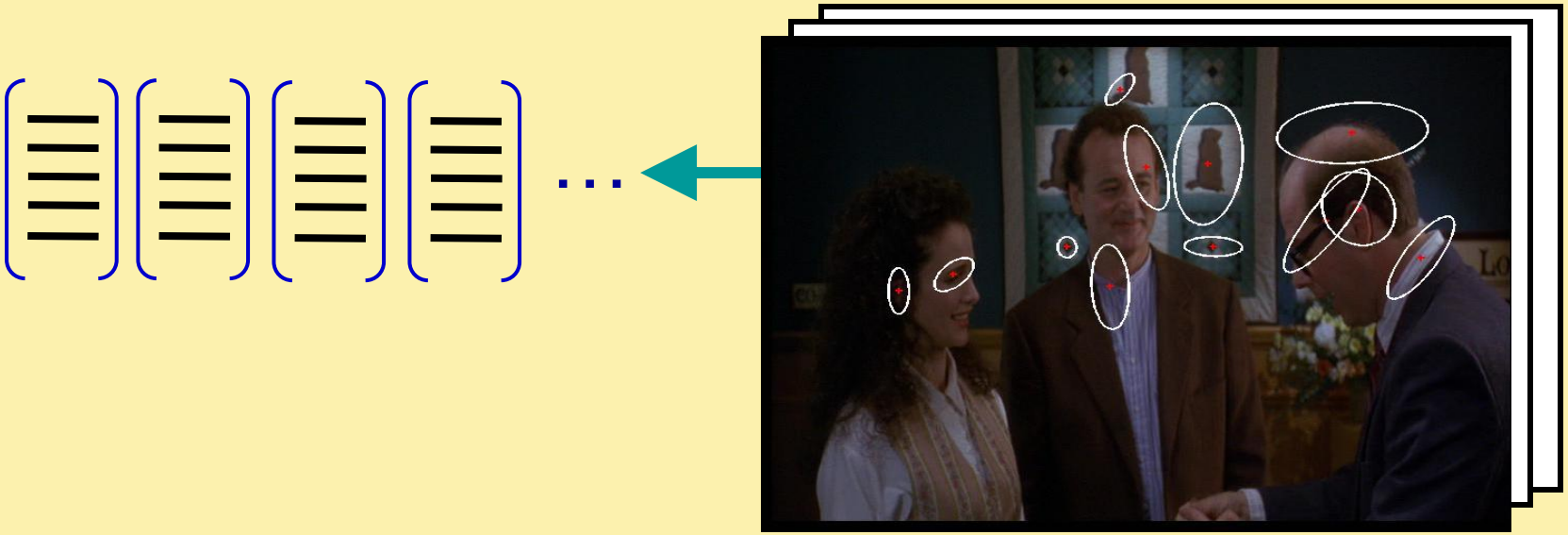
[Mikojaczyk and Schmid '02]

[Mata, Chum, Urban & Pajdla, '02]

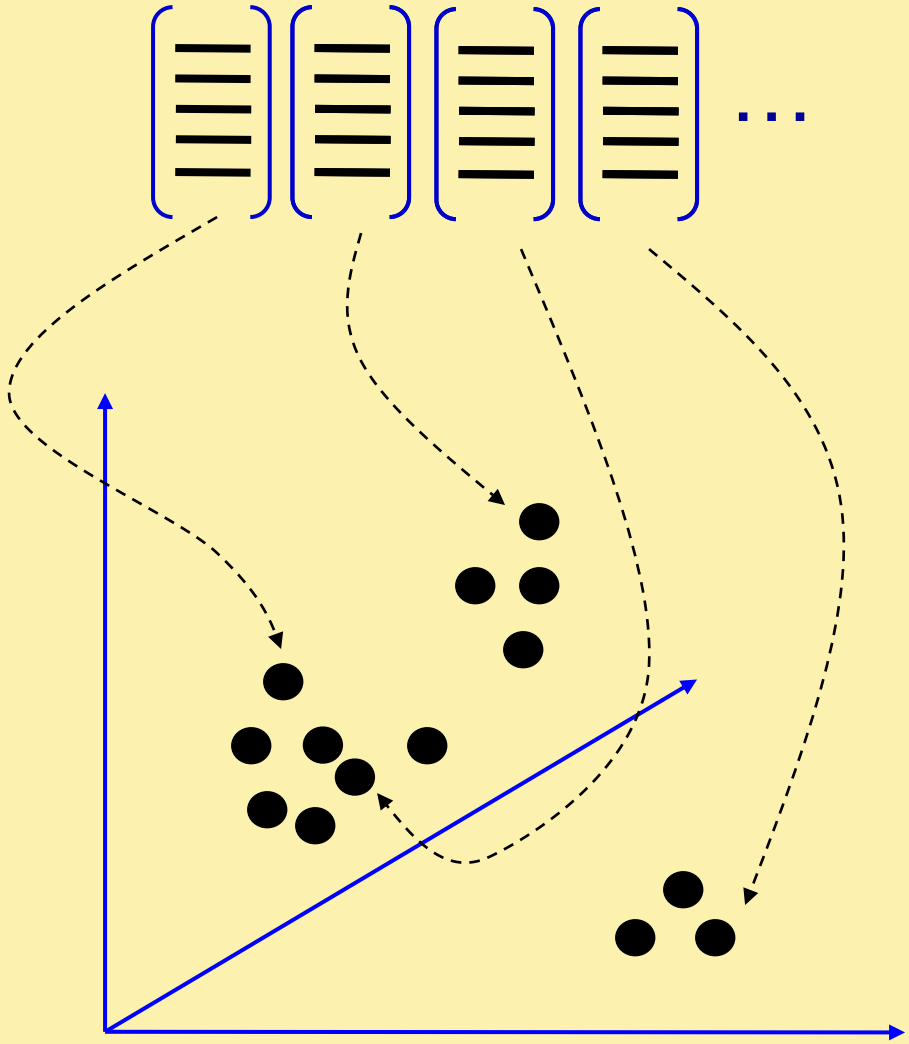
[Sivic & Zisserman, '03]

Slide credit: Josef Sivic

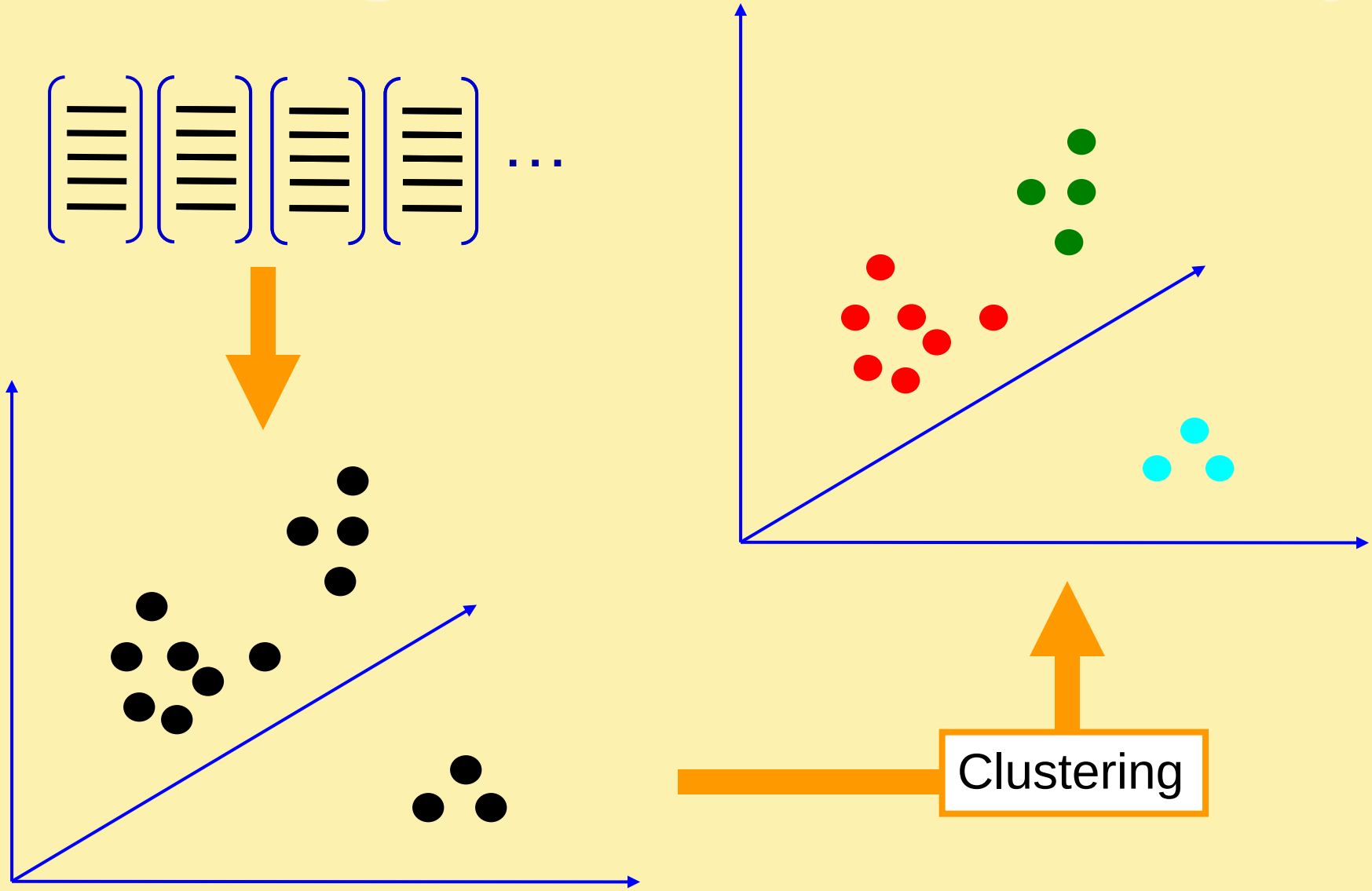
1. Feature extraction



2. Learning the visual vocabulary

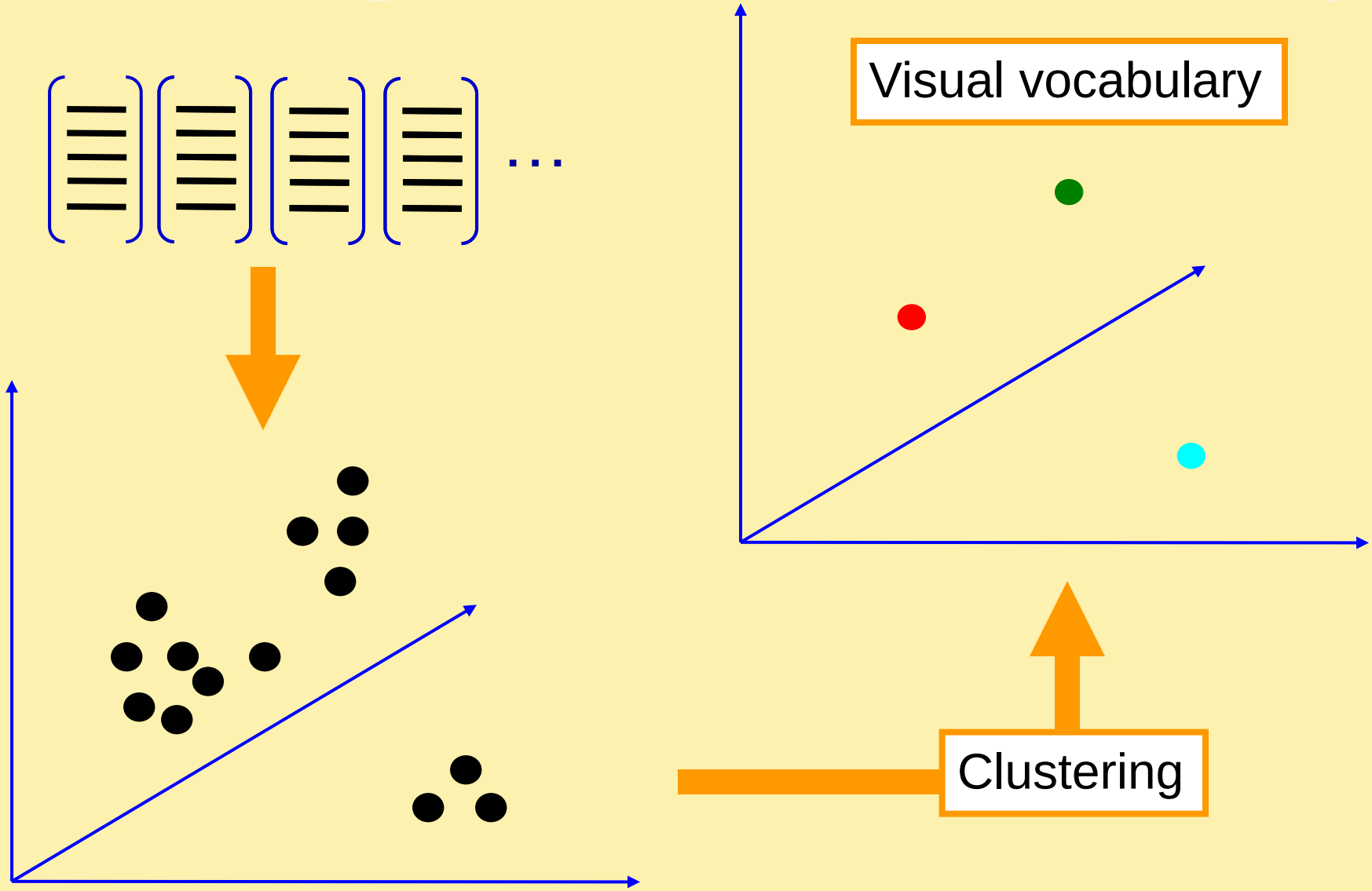


2. Learning the visual vocabulary



Slide credit: Josef Sivic

2. Learning the visual vocabulary



Slide credit: Josef Sivic

K-means clustering

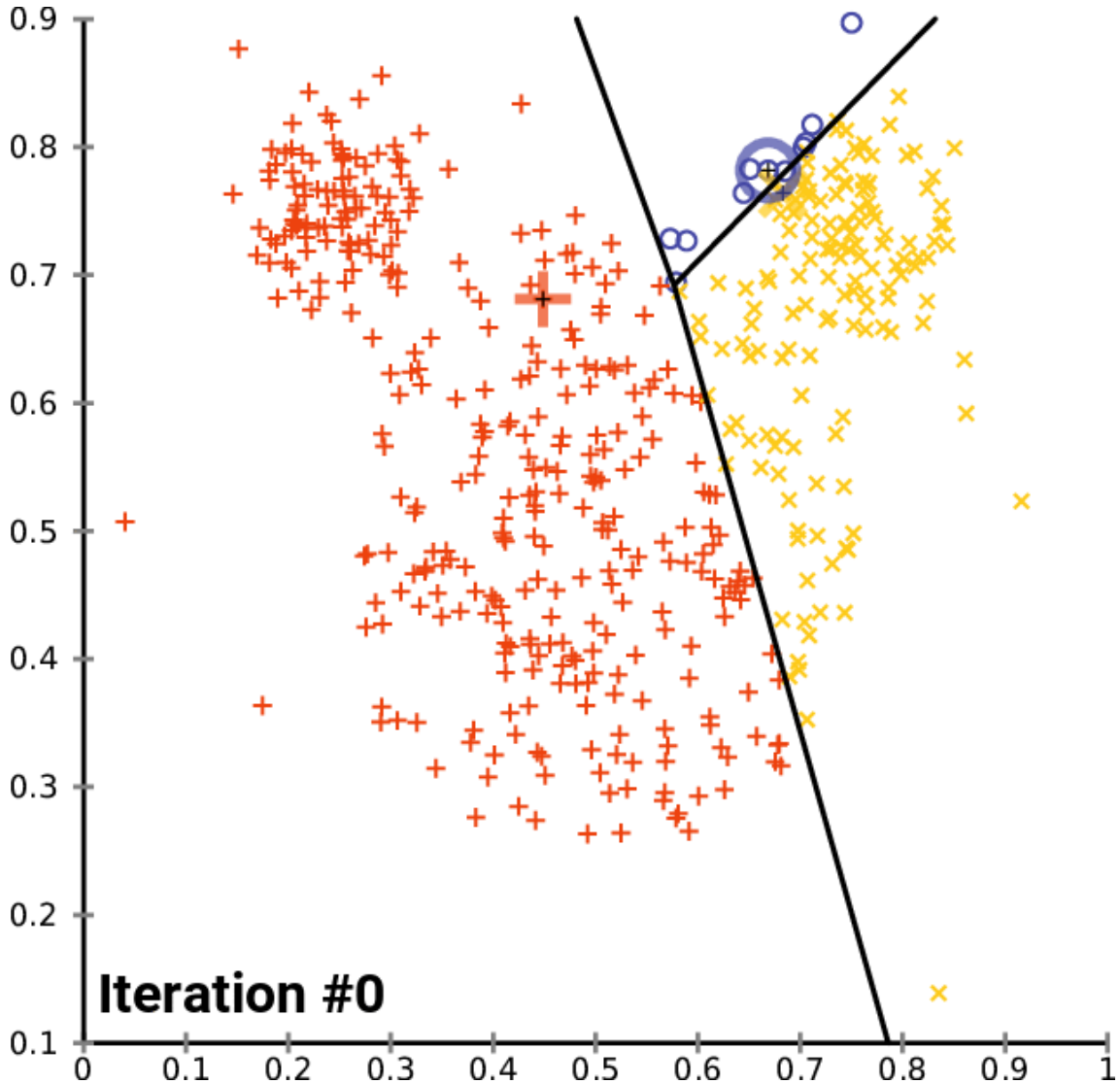
- Want to minimize sum of squared Euclidean distances between points x_i and their nearest cluster centers m_k

$$D(X,M) = \sum_{\text{cluster } k} \sum_{\substack{\text{point } i \text{ in} \\ \text{cluster } k}} (x_i - m_k)^2$$

Algorithm:

- Randomly initialize K cluster centers
- Iterate until convergence:
  - Assign each data point to the nearest center
  - Recompute each cluster center as the mean of all points assigned to it

K-means clustering



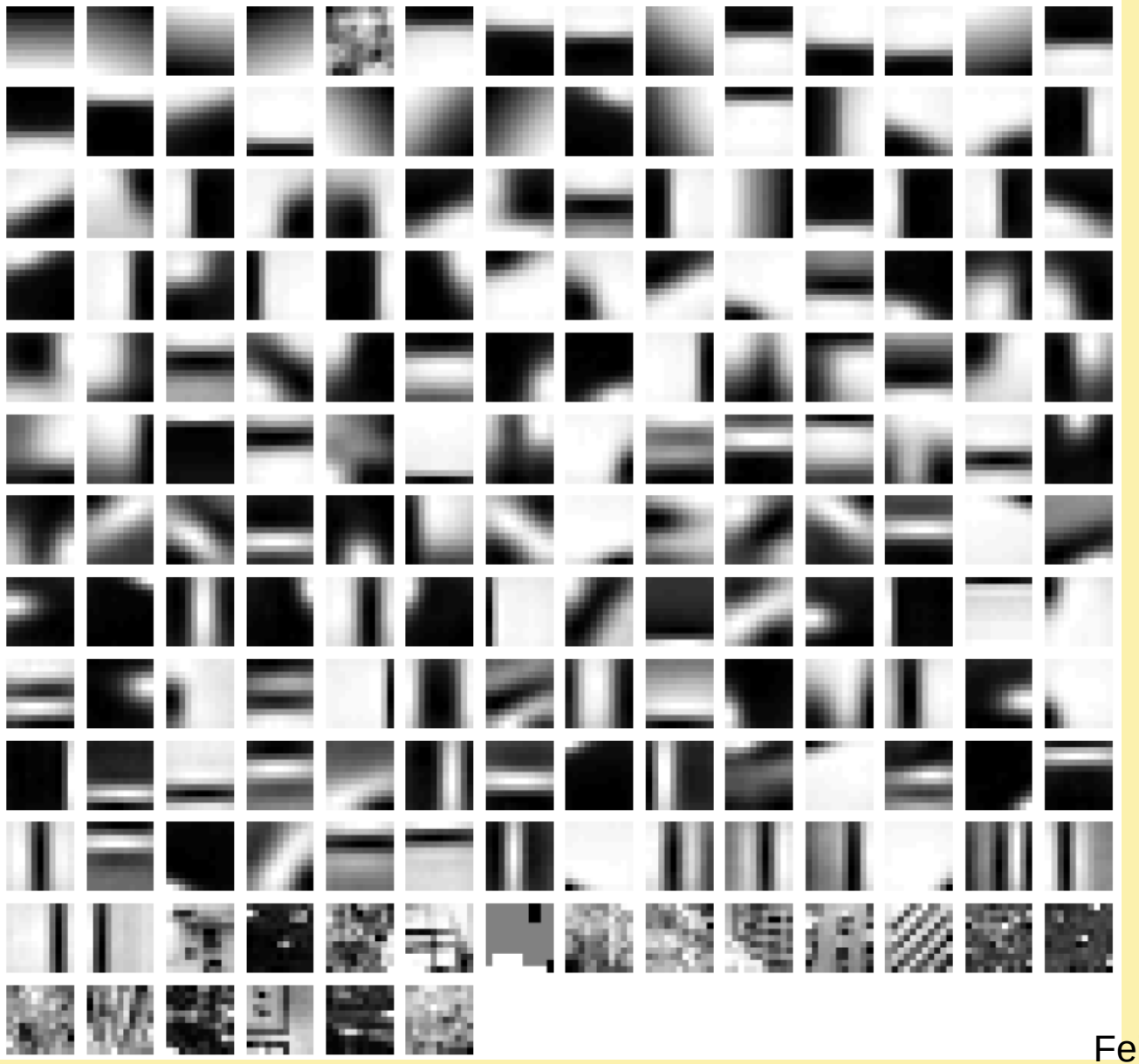
https://en.wikipedia.org/wiki/
File:K-means_convergence.gif

From clustering to vector quantization

- Clustering is a common method for learning a visual vocabulary or codebook
  - Unsupervised learning process
  - Each cluster center produced by k-means becomes a codevector
  - Codebook can be learned on separate training set
  - Provided the training set is sufficiently representative, the codebook will be “universal”

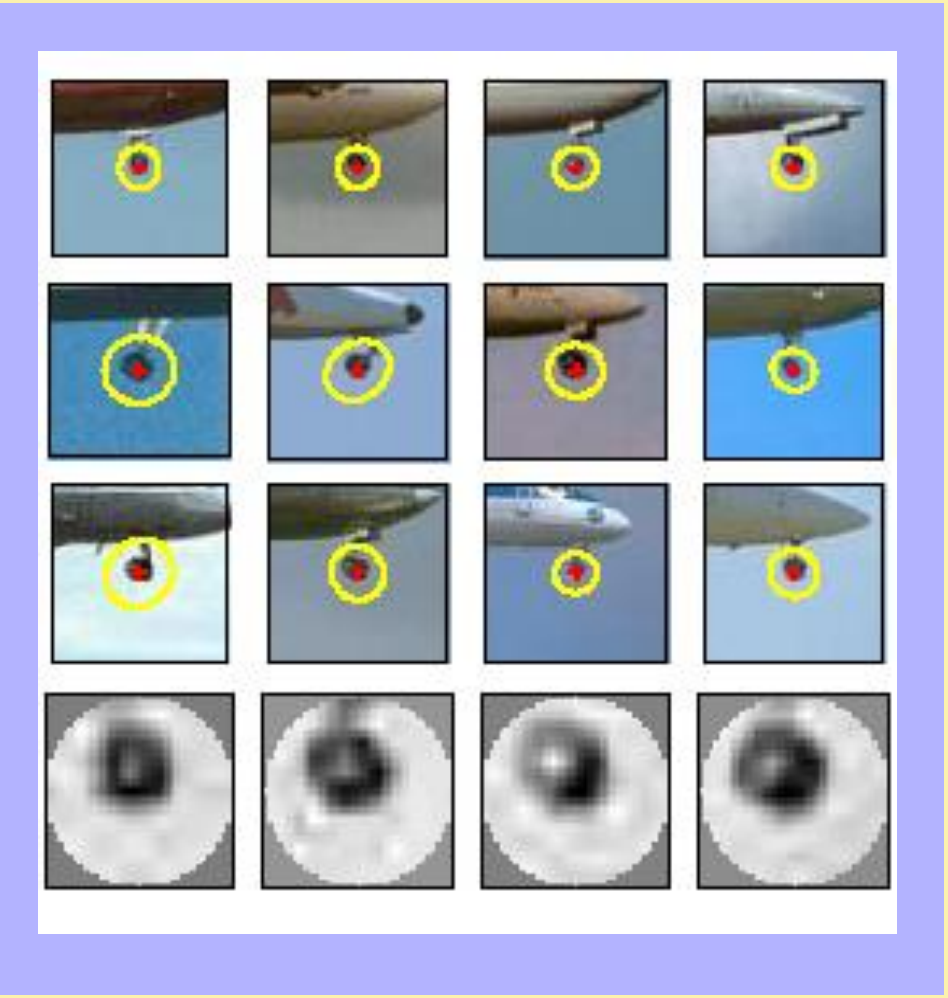
- The codebook is used for quantizing features
  - A *vector quantizer* takes a feature vector and maps it to the index of the nearest codevector in a codebook
  - Codebook = visual vocabulary
  - Codevector = visual word

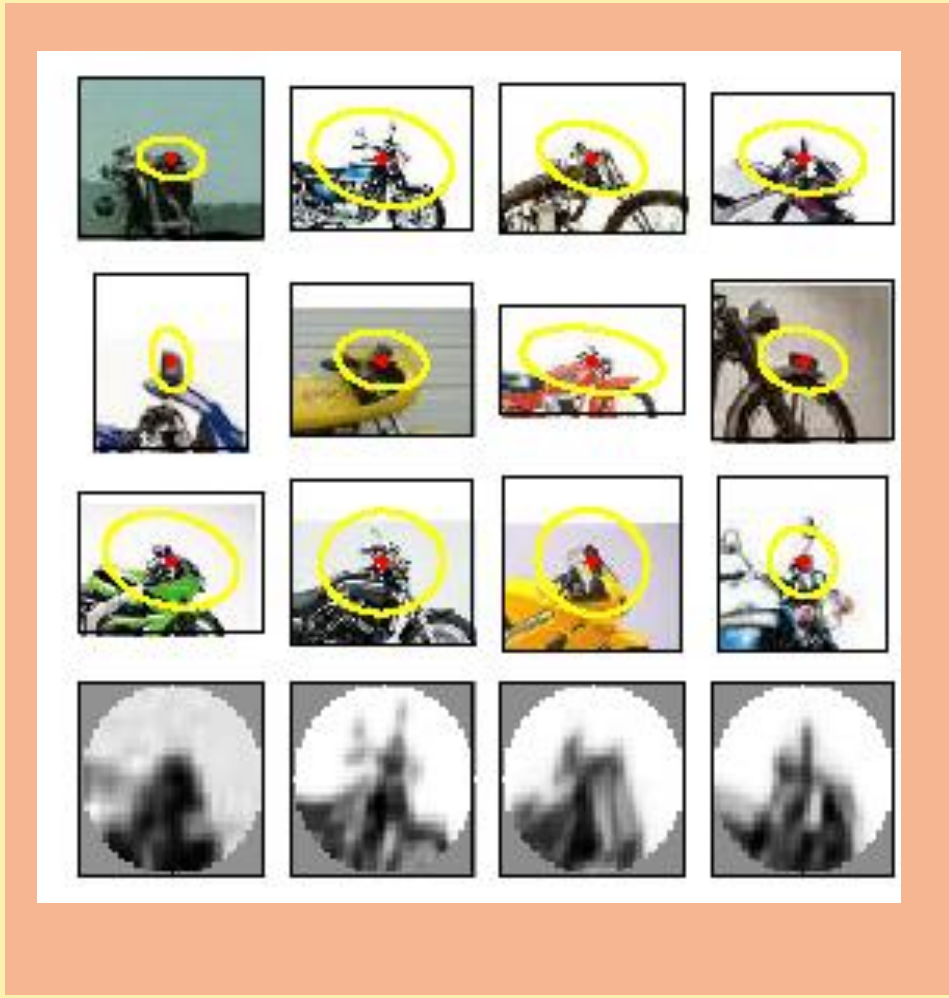
Example visual vocabulary



Fei-Fei et al. 2005

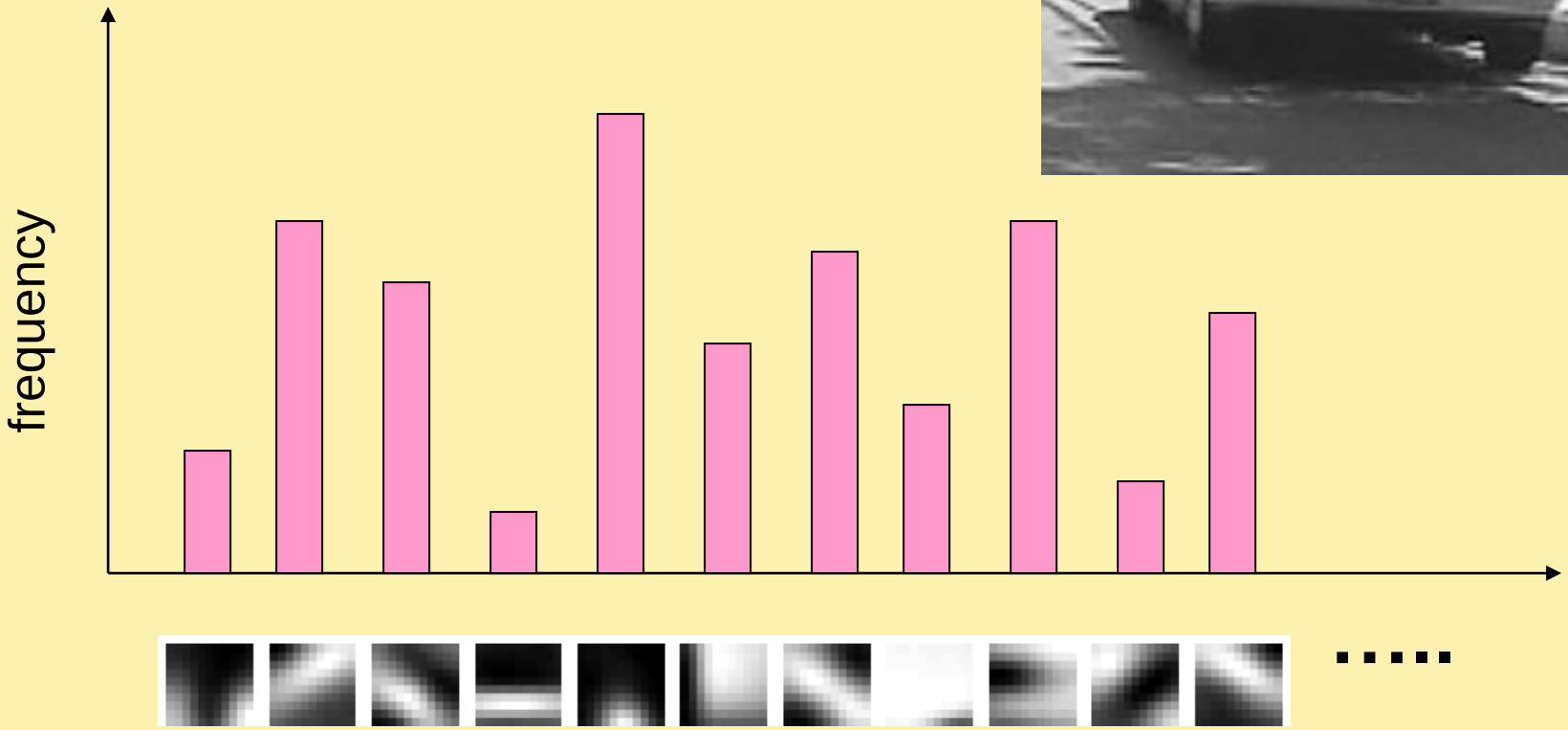
Image patch examples of visual words





Sivic et al. 2005

3. Image representation



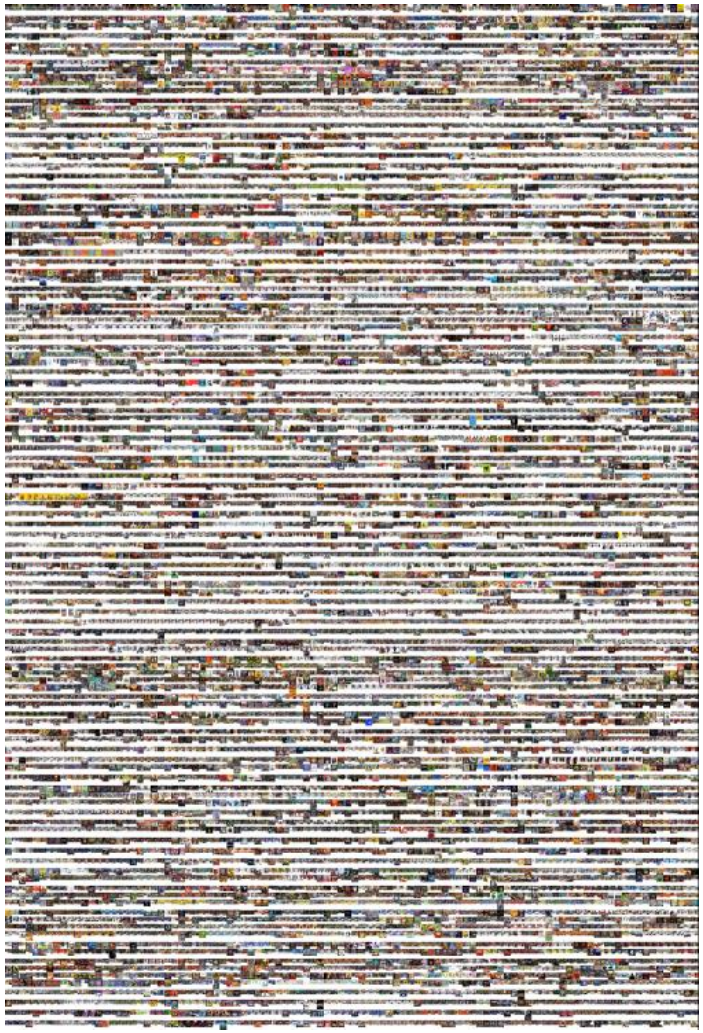
Large-scale image matching

11,400 images of game covers
(Caltech games dataset)

- Bag-of-words models have been useful in matching an image to a large database of object *instances*

how do I find this image in the database?

Large-scale image search



- Build the database:
  - Extract features from the database images
  - Learn a vocabulary using k-means (typical k: 100,000)
  - Compute *weights* for each word
  - Create an inverted file mapping words → images

Weighting the words

- Just as with text, some visual words are more discriminative than others

the, and, or vs. ***cow, AT&T, Cher***

- the bigger fraction of the documents a word appears in, the less useful it is for matching
  - e.g., a word that appears in *all* documents is not helping us

TF (term frequency)-IDF(inverse document frequency) weighting

- Instead of computing a regular histogram distance, we'll weight each word by it's *inverse document frequency*

- inverse document frequency (IDF) of word *j* =

$$\log \frac{\text{number of documents}}{\text{number of documents in which } j \text{ appears}}$$

TF-IDF weighting

- To compute the value of bin *j* in image *I*:

term frequency of *j* in *I* **x** *inverse document frequency* of *j*

TF-IDF weighting

Dataset

document1	*document2*	*document3*	*document4*	*document5*
my dog has flea problem help my dog	maybe not take him to park stupid	my dog is so cute I love him	don‘t buy dog food	it is a banana

Example1

*For word ‘**dog**’ in document 1:*

TF(Term frequency) = 2/8 = 0.25

IDF(Inverse document frequency) = log(5/(3+1)) = 0.097

TF-IDF = 0.25*0.097 = 0.02425

Avoid zero denominator

document1	document2	document3	document4	document5
my dog has flea problem help my dog	maybe not take him to park stupid	my dog is so cute I love him	don‘t buy dog food	it is a banana

Example2

*For word ‘**flea**’ in document*

TF(Term frequency) = 1/8 = 0.125

IDF(Inverse document frequency) = log(5/(1+1)) = 0.398

TF-IDF = 0.125*0.398 = 0.04975

Analysis: *‘Dog’ appears twice while ‘flea’ appears once in document 1. However, ‘dog’ also appears in document 3 and 4. Thus, ‘dog’ is less discriminative than ‘flea’. (weight of ‘dog’(0.097) < weight of ‘flea’(0.398).)*

Inverted file

- Each image has ~1,000 features
- We have ~1,000,000 visual words

  →each histogram is extremely sparse (mostly zeros)

- Inverted file
  - mapping from words to documents

```
"a":      {2}
"banana": {2}
"is":     {0, 1, 2}
"it":     {0, 1, 2}
"what":   {0, 1}
```

Inverted file

- Can quickly use the inverted file to compute similarity between a new image and all the images in the database
  - Only consider database images whose bins overlap the query image

Inverted file

and	1			only	2		
aquarium	3			pigmented	4		
are	3	4		popular	3		
around	1			refer	2		
as	2			referred	2		
both	1			requiring	2		
bright	3			salt	1	4	
coloration	3	4		saltwater	2		
derives	4			species	1		
due	3			term	2		
environments	1			the	1	2	
fish	1	2	3	4	their	3	
fishkeepers	2			this	4		
found	1			those	2		
fresh	2			to	2	3	
freshwater	1	4		tropical	1	2	3
from	4			typically	4		
generally	4			use	2		
in	1	4		water	1	2	4
include	1			while	4		
including	1			with	2		
iridescence	4			world	1		
marine	2						
often	2	3					

Fish in water?

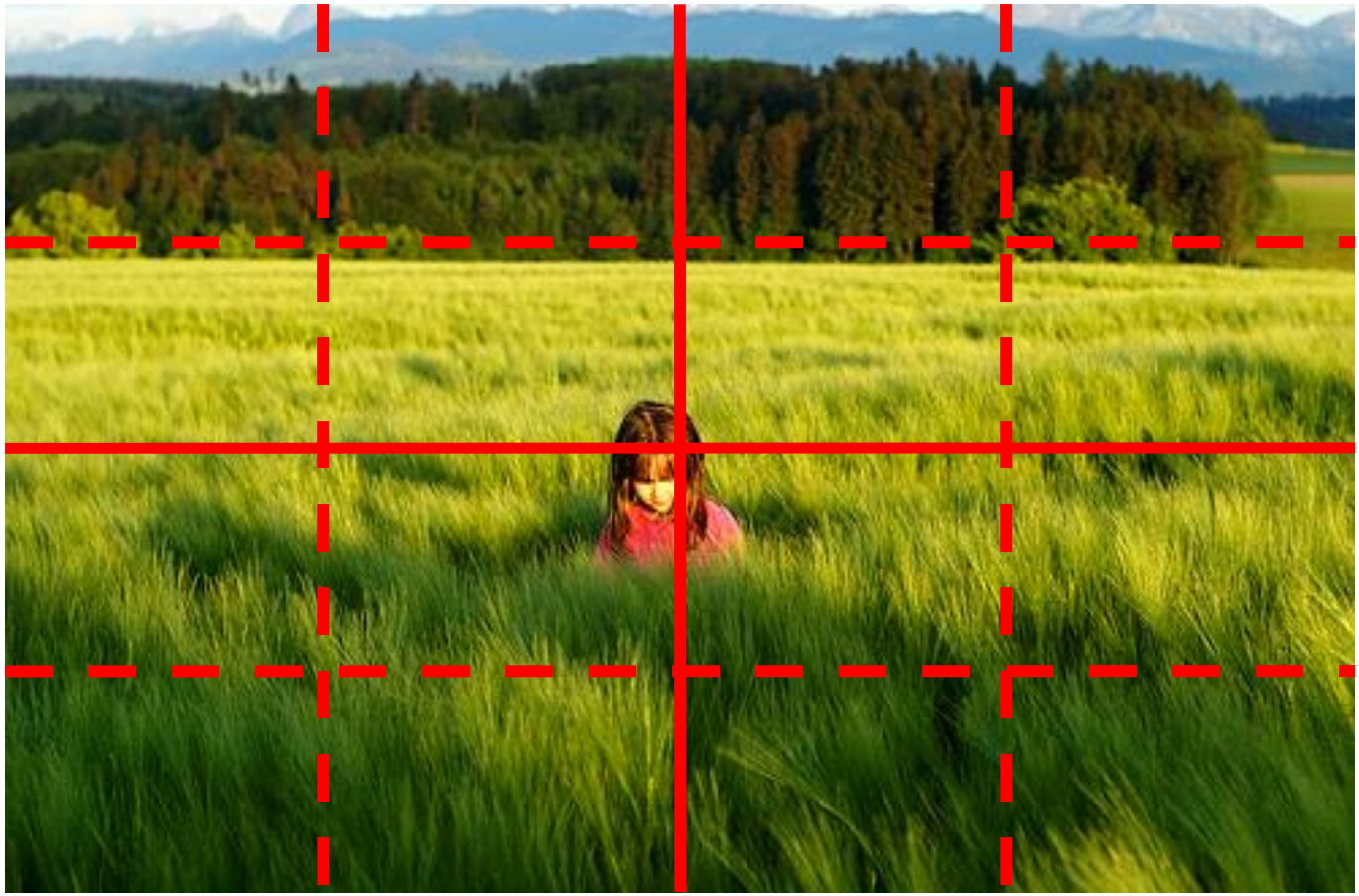
Inverted file

File id	Content
1	I like math
2	I like programming
3	programming is difficult

Mapping from words to documents:

item	File id
I	1, 2
like	1, 2
math	1
programming	2, 3
is	3
difficult	3

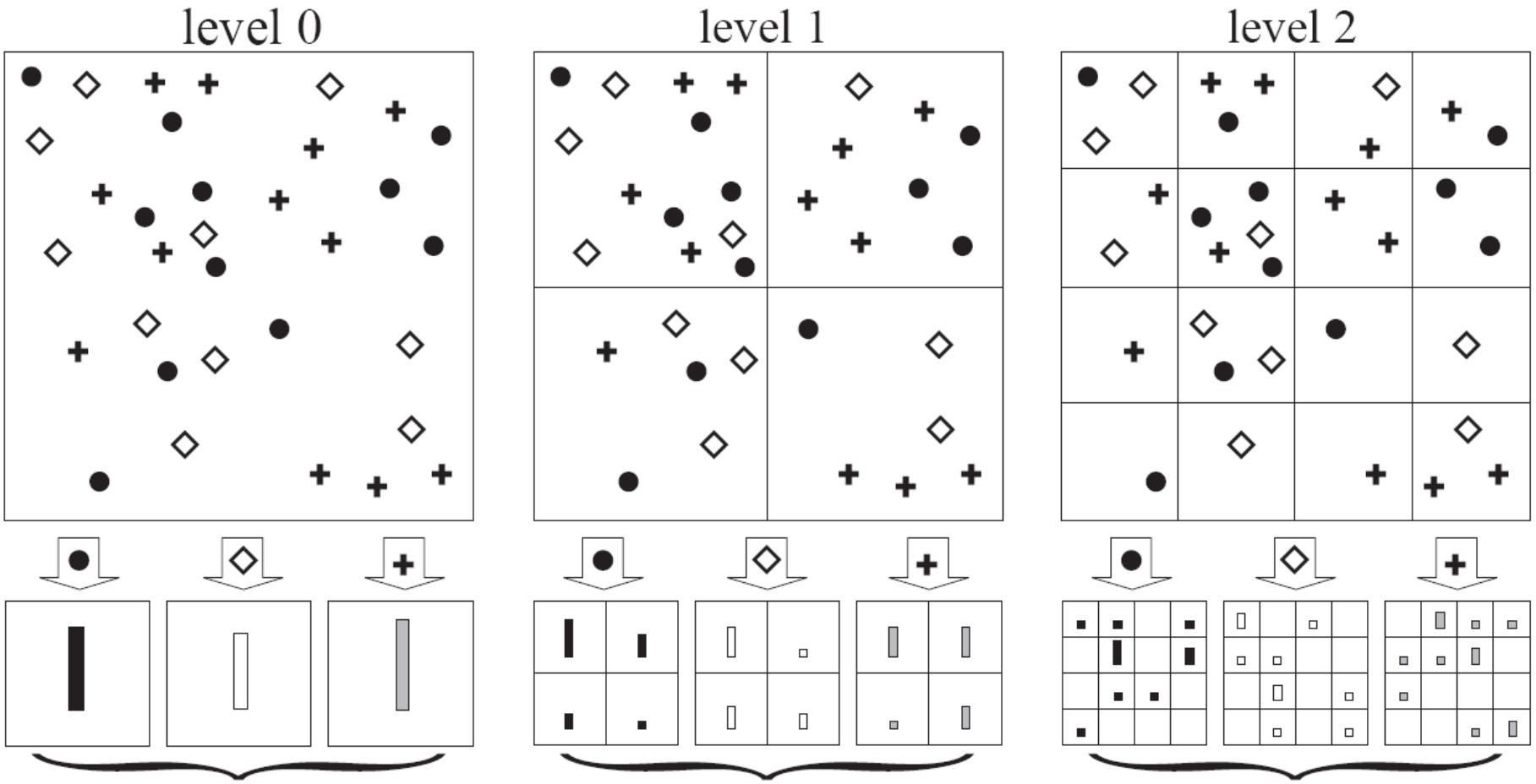
Spatial pyramid: BoW disregards all information about the spatial layout of the features



Compute histogram in each spatial bin

Slide credit: D. Hoiem

Spatial pyramid



[Lazebnik et al. CVPR 2006]

Slide credit: D. Hoiem